Infrastructure Engineer Assignment

Infrastructure Engineer | SRE / DevOps — Broadcasting

Ali Alkhathami

August 2026

github.com/Alkhathami1/devops-tasks

Contents

1. Summary

Six infrastructure tasks, each built and then run against a real runtime: a Windows-to-Linux SMB mount with monitoring and alerting, a multi-tier Docker stack on isolated networks, a SQL Server database with tiered backups and point-in-time recovery, an S3 multipart copy engine driven from Node-RED, a three-tier GCP network built with Terraform and configured by Ansible, and an HEVC contribution pipeline archived to S3 by a live AWS Elemental MediaLive channel.

The logs in `docs/evidence/` carry the command, the timestamp and the full output behind every figure in this report. Each task directory holds a `WALKTHROUGH.md` with the full reasoning and the complete steps.

Six results that carry furthest.

A multipart copy whose destination ETag ends `-5` against a single-PUT source ETag with no suffix. The suffix is the part count, so the two ETags are direct evidence that the copy used the multipart API rather than a single operation.

A container held at `OOMKilled=true` with exit 137, sampled at t+1020ms, with the blast radius contained: the cgroup limit stopped the memory hog, the restart policy replaced the container, and the two sibling services never noticed.

A point-in-time restore that landed between two committed transactions — marker A present, marker B absent — recovering to an instant chosen after the damage, not to a backup boundary.

`ffprobe` on a segment MediaLive itself produced, reporting `Video: hevc (Main) ([36][0][0][0] / 0x0024)` at 1920x1080 with AAC at exactly 192 kb/s. Stream type `0x24` is HEVC's native MPEG-TS identifier, read off the file AWS wrote.

Three Prometheus alert rules driven from inactive through pending to firing on live conditions and back again, including one watching the SMB share itself.

A Windows client mapping a Linux Samba share across a private WireGuard tunnel at SMB dialect 3.1.1 encrypted, with the file it wrote read back on the server's own filesystem — and the same address carrying no route at all before the tunnel was raised.

---

2. Task 1 — Cross-platform volume mounting

> Mount a volume from Windows to Linux and explain: protocol used and why, mounting steps, making it persistent across reboots, monitoring mount and disk performance, alerting to guarantee service level, create a filesystem handling single files up to 1 TB with justification, and the reverse direction, Linux to Windows.

Decisions

SMB 3.1.1, on the identity model rather than throughput. NFSv3 maps identity by numeric uid and gid, so the client asserts who it is and the server believes it — worthless across a Windows boundary where the two sides hold unrelated user databases. NFSv4 improves on that with string principals but needs `idmapd` configured consistently on both ends plus Kerberos for real authentication. SMB authenticates the session before any file access and Windows ACLs map onto it natively, because it is Windows' own protocol. iSCSI was rejected as the wrong layer entirely — it exports a block device, and two hosts mounting one LUN without a cluster filesystem corrupt it. sshfs is single-threaded FUSE with no ACL model.

XFS for the 1 TB requirement. ext4's default configuration caps a single file at 16 TiB, which clears 1 TB but with the ceiling in sight; XFS is extent-based with a far higher limit, and its allocation groups let large files grow without the fragmentation a bitmap allocator produces. ZFS was rejected for the out-of-tree module and the memory footprint.

What was built and run

A container serving Samba with `server min protocol = SMB3_11` and `smb encrypt = required`, running node_exporter alongside it in the same mount namespace, with Prometheus scraping both. A real Windows share, created on the host with a dedicated local account, mounted on Linux with `mount.cifs` using a mode-600 credentials file. XFS built on a loop device. Alert rules written against a stated availability, latency and capacity objective.

The round trip is the proof: Linux wrote a timestamped file to the Windows share, and Windows read it back byte for byte with `Get-Content`.

Measurement	Value	Evidence
Negotiated dialect	0x311, session encrypted	01-direction-a.log
Write to the share, fsync	85.1 MB/s	01-direction-a.log
Cold read over SMB	26.0 MB/s	01-direction-a.log
Read with O_DIRECT	24.3 MB/s	01-direction-a.log
XFS single-file ceiling	8 EiB, measured	01-xfs.log
1 GiB file extent map	3 records across 3 allocation groups	01-xfs.log
Alert rules loaded / driven to firing	10 / 3	01-alerts.log
Persistence across a reboot	mount live at container start, share readable, no manual command	01-reboot.log
Full suite	8 checks, 0 failures	01-verify-suite.log

Insights

A read taken straight after a write measures the page cache. The first throughput run reported 9.5 GB/s over SMB. The tell was not that it looked good — it was that 9.5 GB/s is about 76 times what a 1 GbE link can carry, so the number could not describe the network. Dropping caches and repeating gives 26.0 MB/s, and an `O_DIRECT` read confirms it at 24.3 MB/s. Every throughput figure now names the layer it describes.

A filesystem metric cannot see a reconnecting SMB session. node_exporter reports a CIFS mount as size, free and readonly, which answers "is it mounted and does it have room" but not "is the client re-establishing the session on every operation". The session and reconnect counters in `/proc/fs/cifs/Stats` answer that, and they rise before throughput falls, so a textfile-collector export of them is the leading indicator for the share.

node_exporter's diskstats collector excludes loop devices by default. The XFS filesystem under test lives on one, so every disk metric for it was absent while the exporter itself looked healthy — no error, just missing series.

Full detail in `01-storage-mounts/WALKTHROUGH.md`*; evidence in* `01-*.log`*.*

---

3. Task 2 — Containerized multi-tier application

> Multiple applications communicating over a private Docker network — database container, backend container, frontend and reverse proxy container. Automate as much as possible, use a secret manager or environment variables, explain how the system recovers from unexpected failures such as machine shutdown and RAM exhaustion, and explain how CPU, storage and RAM are allocated.

Decisions

Two networks, not one. The task asks for a private network; two make the boundary structural. `edge-net` carries nginx to the backend, `internal-net` carries the backend to PostgreSQL and is marked `internal: true`, which removes its default route entirely. nginx has no interface on it, so nginx cannot resolve `postgres`, let alone reach it.

PostgreSQL rather than MSSQL, because Task 3 covers SQL Server with its own container and Postgres keeps this stack light enough for a meaningful memory exhaustion drill.

Both secret mechanisms, so the difference can be shown. The database password is a Docker file-based secret; non-sensitive configuration sits in `.env`.

What was built and run

A three-service Compose stack brought to healthy by one command, with healthchecks and `depends_on: service_healthy`, and migrations and seed running idempotently at startup. Five failure drills, each executed against the live stack.

Drill	Result	Evidence
Backend crash	restart policy replaced it, RestartCount 0 to 1	02-drill-a-crash.log
RAM exhaustion	OOMKilled=true, exit 137, sampled t+1020ms	02-drill-b-oom.log
Docker daemon restart	all three services returned unattended	02-drill-c-daemon-restart.log
Volume persistence	data survived down then up; down -v removed it	02-drill-d-persistence.log
Database outage	/health 503 in 2.00 s, reconnected in 1503 ms, no crash-loop	02-drill-e-dbfail.log
Network isolation	19 checks, nginx cannot resolve postgres	02-network-isolation.log
Allocation	2.25 CPU and 896 MiB of limits against a 12-CPU, 7.7 GiB VM	02-resources.log

Insights

`docker kill` does not trigger a restart policy. Docker records it as operator intent, and `unless-stopped` deliberately declines to act on it. A drill written around `docker kill` exercises administrative stop, not crash recovery. Proving recovery needs PID 1 to exit non-zero on its own.

`depends_on: service_healthy` is compose-time ordering only. On a daemon restart the guarantee is gone and everything comes back at once — the backend started 15 ms before PostgreSQL. The stack still converged, but it converged because the application retries its database connection. Credit belongs to the retry loop, not to Compose.

`OOMKilled` must be sampled at the moment of the kill. The replacement container carries a fresh state object with the flag false, so a check that inspects after recovery finds nothing and reports a clean run.

An environment variable is visible to anyone who can reach the Docker socket. `docker inspect` prints it in plaintext without entering the container; the file-based secret shows only the path `/run/secrets/`. Both mechanisms work, and only one fails safe.

Full detail in `02-docker-stack/WALKTHROUGH.md`*; evidence in* `02-*.log`*.*

---

4. Task 3 — SQL Server administration

> Set up an MSSQL database, managed or self-hosted, and create a database, create tables and insert data, take automated backups, and restore from a backup.

Decisions

Self-hosted SQL Server 2022 in Docker, pinned by tag and digest. A managed service would hide exactly the backup mechanics the task asks to demonstrate: Azure SQL and RDS own the backup schedule, the retention window and the restore path, and expose them as settings rather than as operations.

FULL recovery model, and three backup tiers. Recovery model is the decision that makes point-in-time recovery possible at all. A full backup is the base, a differential shortens the replay, and log backups set the recovery point — a restore needs the last full, the most recent differential, and only the logs since that differential.

What was built and run

A supervised sidecar takes an unattended full backup at startup so a restore base exists immediately, then runs the three tiers on their own intervals with a retention pass. `msdb.dbo.backupset` records the cadence with nobody driving it: six log backups five minutes apart and a differential in the middle. Every backup is written `WITH CHECKSUM` and verified with `RESTORE VERIFYONLY` before it is counted, and the script exits non-zero if verification fails. Two restore drills followed: a full round trip after schema-level damage, and a point-in-time recovery.

Measurement	Value	Evidence
Schema	4 tables, 3 foreign keys, 7 check constraints, 11 indexes	03-schema.log
Constraint enforcement	orphan insert rejected by FK_orders_customer	03-schema.log
Scheduler cadence, unattended	6 log backups at 5-minute intervals, differential at 02:47:43	03-verify-suite.log
Full backup compression	8.2x	03-backup-tiers.log
Restore, engine time	554 pages in 0.050 s (86.484 MB/sec)	03-restore-roundtrip.log
Restore, end to end	2.09 s	03-restore-roundtrip.log
Post-restore fingerprint	identical to pre-damage	03-restore-roundtrip.log
Point-in-time recovery	marker A present, marker B absent	03-point-in-time.log

Insights

Recovery time is two numbers, and they differ by a factor of forty. SQL Server reports the restore itself at 0.050 s. The drill measures 2.09 s, because its timer wraps `SET SINGLE_USER`, the restore and `SET MULTI_USER` — three separate `docker exec` calls, each paying sqlcmd startup. The engine figure is the one that scales with database size; the end-to-end figure is the one an operator plans against, because the tooling around a restore is part of the outage.

SQL Server Agent ships present in the Linux container, merely disabled. The common claim is that it is unavailable there. The Developer image carries it and `MSSQL_AGENT_ENABLED=true` brings it up, confirmed by probing the instance.

A test worth trusting has been seen to fail for the right reason. A restore drill compared a data fingerprint before and after. Both captures had picked up sqlcmd's `Changed database context to 'AppDb'.` banner rather than any data, so the comparison was between two identical banners and would have passed whatever the restore did. Filtering the banner and re-running is what turned it into evidence.

Full detail in `03-mssql/WALKTHROUGH.md`*; evidence in* `03-*.log`*.*

---

5. Task 4 — S3 multipart copy through Node-RED

> Copy a file between S3 buckets using multi-part upload, raise the upload size limit to the maximum, and add appropriate logging.

Decisions

`UploadPartCopy`, not GetObject then PutObject. The server-side range copy keeps object bytes inside S3 entirely: the client sends a byte range per part and S3 moves the data. A download-then-upload flow would stream every byte through Node-RED, bounding the copy by the machine's link speed and its memory.

Part size defaults to the 5 GiB maximum, which is what "raise the upload size limit to the maximum" asks for, with an auto-raise when an object would otherwise need more than 10,000 parts.

What was built and run

A copy engine with bounded-concurrency workers and exponential backoff with jitter, calling `AbortMultipartUpload` on every failure path so a failed copy leaves no in-progress upload accruing storage. A Node-RED flow drives it. The suites ran against moto, a local S3-compatible server, so the API is real while the account is not.

Measurement	Value	Evidence
Unit tests	19 of 19	04-unit-tests.log
Integration tests against moto	9 of 9	04-integration-moto.log
Destination ETag	66252e6c…-5 against a single-PUT source	04-orphan-check.log
Default part size	5,368,709,120 bytes (5.00 GiB)	04-part-sizing.log
1 TiB plan at the default	205 parts	04-part-sizing.log
1 TiB plan at 5 MiB requested	auto-raised to 105 MiB, 9,987 parts	04-part-sizing.log
Orphaned multipart uploads	0 across both buckets	04-orphan-check.log

Insights

Raising the part size to its maximum removes the behavior the task asks to show. At 5 GiB a 23 MiB object is one part, and the completed upload carries an ETag ending `-1`. Both requirements are satisfiable and not in the same run: the maximum is proven by the planner and by a live copy at that setting, and multipart is proven by deliberately choosing a smaller part size so the object spans five parts and the ETag reads `-5`.

The ETag suffix is the part count, which makes it a free integrity signal. A single-PUT object's ETag is the MD5 of its content with no suffix; a multipart object's is a digest of the part digests followed by `-N`. Comparing source and destination ETags therefore says which API path each object took.

A default is a decision about what happens when someone forgets. The endpoint resolver defaults to the local moto endpoint, and reaching real AWS requires setting `S3_ENDPOINT=aws` explicitly. Every tool prints the resolved target before doing any work, so which account is being touched is never a guess.

Full detail in `04-nodered-s3/WALKTHROUGH.md`; evidence in `04-*.log`.

6. Task 5 — Three-tier infrastructure as code

> Three networks — public with an nginx reverse proxy routing to the app, apps with an app server that reads from and writes to the database, and dbs with the database — in a three-tier architecture with proper network segmentation and access to the servers over a private VPN, especially the app and DB tiers.

Decisions

Three separate VPCs, not three subnets in one. The requirement says networks, and separate VPCs make the tier boundary structural. GCP VPC peering is non-transitive: peering public to apps and apps to dbs does not give public to dbs. The public tier therefore has no route to the database range at all, so a misconfigured firewall rule cannot expose it — the packet has nowhere to go.

Deny-by-default firewalls with service accounts as sources, not IP allowlists, so a rebuilt instance keeps its authorization without an address update.

No public IPs on the app or database instances. Egress for package installation runs through Cloud NAT.

What was built and run

Terraform describes 32 resources: three VPCs with peering, subnets, ten firewall rules, four instances, Cloud NAT and routers. Ansible configures them through a dynamic inventory built from `terraform output -json`, reaching the private hosts by `ProxyJump` through the bastion. WireGuard is deployed on that bastion with a generated keypair, the interface up on `10.99.0.1` and the client peer configured.

The end-to-end proof is a row written through nginx to the app to the database and read back on a second request.

Measurement	Value	Evidence
Resources planned	32 to add, 0 to change, 0 to destroy	05-plan.log
Architecture checks	17 of 17	05-architecture.log
Tier traversal	row written through nginx to app to db, read back	05-architecture.log
Public addresses	2 of 4 instances	05-architecture.log
Route to the database range from public	none exists	05-architecture.log
Tunnel handshake	completed, 900 B received / 764 B sent	05-vpn-smb.log
Through the tunnel	ICMP to 10.20.1.2 at ~212 ms, TCP 445 open	05-vpn-smb.log
Same address, tunnel down	no adapter, no route, 445 refused	05-vpn-smb.log
Windows client mapping	Z: to the app-tier share, dialect 3.1.1, encrypted	05-vpn-smb.log
Ansible second run	changed=0 on all four hosts	05-ansible-idempotency.log
Post-destroy sweep	every resource class clean	05-destroy-orphan-check.log

Insights

Non-transitive peering is a feature, not an obstacle. It enforces the tier boundary at the routing layer rather than by rule, which is why the WireGuard client configuration advertises only the ranges it can actually reach — advertising the database range would blackhole those packets rather than deliver them.

Network tags and service accounts do not traverse VPC peering. A firewall rule in one VPC cannot name a service account in another as its source, so cross-VPC rules have to be expressed by range while intra-VPC rules keep the service-account form.

A destroy command reporting success is not the same as the project being empty. It knows only what is in its own state. The sweep afterwards queries GCP directly, per resource class, and covers the classes that outlive an instance: reserved addresses, disks, NAT gateways, snapshots and images.

Full detail in `05-terraform-ansible/WALKTHROUGH.md`*; evidence in* `05-*.log`*.*

---

7. Task 6 — HEVC contribution and MediaLive archive

> At least Full HD, at least 12 Mbps video and 192 kbps audio, optionally justify a different bitrate with an explicit BPP formula, HEVC codec, and record the stream to S3 as .mxf or .ts, documenting every step.

Decisions

12 Mbps at 1080p60, defended by bits per pixel. BPP is bitrate divided by width times height times frame rate, which normalizes bitrate against how much picture is being carried per second. 12,000,000 / (1920 x 1080 x 60) = 0.0965 bpp, mid-band for live HEVC, where roughly 0.05 to 0.15 is the sane range. The same 12 Mbps is 0.1929 bpp at 1080p30 and 0.0241 bpp at 4K60 — a quarter the value, and below the band.

MPEG-TS over MXF for the archive. TS is segmentable, so a damaged region costs those packets and the stream resynchronizes at the next alignment point; a damaged MXF header or index can lose the file. TS is also what MediaLive's ARCHIVE output group writes natively.

RTMP push as the contribution input, with HEVC at the deliverable. OBS speaks RTMP natively from its own Stream output, so the contribution leg is H.264 over RTMP; MediaLive decodes it and encodes HEVC into the ARCHIVE output group, which makes the `.ts` segments in S3 — the thing the task asks to record — HEVC. The split is deliberate rather than a compromise: classic RTMP has no CodecID for HEVC, Enhanced RTMP adds one and ffmpeg muxes it without complaint, and MediaLive's RTMP input still expects H.264. The constraint belongs to the receiver, so the codec requirement is met where the requirement actually lives.

What was built and run

Local HEVC encodes with ffmpeg from a synthetic source, so the whole analysis is reproducible, measured with ffprobe. A codec comparison against H.264 scored with VMAF at two bitrates. Terraform describing the AWS side: an S3 bucket, a MediaLive input security group, an RTMP push input, a single-pipeline channel with an H.265 encode and an ARCHIVE output group, and the IAM role MediaLive needs. OBS Studio 32.2.1 was installed and configured programmatically - profile and scene collection written where OBS reads them, no GUI - and launched with `--startstreaming --startrecording`. The channel took that feed and wrote its archive to S3. The feed itself was blank: OBS's screen-capture source produced no image, so what crossed the link and landed in the archive is valid HEVC carrying a black picture.

Measurement	Value	Evidence
Local encode	1920x1080, hevc Main, 12,652,330 bps overall	06-encode.log
Audio	AAC 194,205 bps locally, exactly 192 kb/s from MediaLive	06-encode.log, 06-s3-archive.log
GOP	2.00 s, counted from frames and keyframes	06-encode.log
BPP at 1080p60, 12 Mbps	0.0965	06-analysis.log
VMAF at 12 Mbps	HEVC 74.686293, H.264 75.093992	06-analysis.log
VMAF at 3 Mbps	HEVC 69.975861, H.264 69.023453	06-analysis.log
OBS contribution	1920x1080, 60/1 fps, RTMP connected, streaming and recording	06-obs.log
MediaLive segment	hevc (Main) … 0x0024, 1920x1080, 60 fps, AAC 192 kb/s	06-s3-archive.log
Archive written	36 .ts segments from the OBS feed	06-s3-archive.log

Measurement	Value	Evidence
Picture carried	none: archived segment black for 99.8% of its duration, luma 16	06-picture-check.log
Post-destroy sweep	channels, inputs, security groups, buckets, roles clean	06-teardown.log

Insights

HEVC's advantage over H.264 is bitrate-dependent, and it crosses over. At the required 12 Mbps for 1080p60, HEVC scored 74.686293 against H.264's 75.093992. At 3 Mbps the order reverses: 69.975861 against 69.023453. The widely quoted 40 to 50 percent saving is a low-bitrate result. At 0.0965 bpp both codecs have budget to spare, and HEVC's extra tools buy little.

Enhanced RTMP carries HEVC, so the container is not the constraint. Classic RTMP/FLV has a 4-bit CodecID with no value for HEVC, and the 2023 Enhanced RTMP specification adds one through a FourCC extension that modern ffmpeg implements — muxing HEVC into FLV succeeds. What actually decides the transport is the receiver: MediaLive's RTMP input expects H.264. Producing a file ffmpeg is happy to mux proves nothing about what a service will accept.

A transport stream's mux rate is not the elementary stream's bitrate. The channel is configured with a 12 Mbps video elementary stream and a constant mux rate of 13,192,000 bps — video plus audio plus a megabit of headroom. A probe of a segment measures the container, so the steady-state segments work out to about 13.67 Mbps against that mux rate, and the first segment reads higher still because it carries the initial signaling. Naming the layer is what keeps the comparison honest.

Every check was green and none of them looked at the picture. OBS logged a successful RTMP connection, a streaming start and a recording start; ffprobe reported `hevc (Main)` at 1920x1080p60 with AAC at 192 kb/s; the bucket listing reported 36 segments. All of that is true, and all of it is true of a black feed, because a blank canvas encodes to valid HEVC at the requested parameters exactly like a photographed one. The suite measured the envelope and never the image, so it could not have failed for the one thing it was carrying. Measuring black duration and sampled luma settles it in a line — luma 16 across every sampled frame, against 126 for the control — and that check now gates `verify-archive.sh`. A check that cannot go red for what you care about is not evidence of it.

Full detail in `06-streaming/WALKTHROUGH.md`*; evidence in* `06-*.log`*.*

---

8. Method

The work ran as a small delegated team: one side builds, the other reviews, and they were kept apart because a pass that writes its own verification produces checks that agree with the code.

Each task was built, run against a real runtime, and its output captured before anything was written about it. Every figure here comes from a log in `docs/evidence/`, written by a wrapper that records the command, the timestamp and the full output, and redacts on the way in.

Review runs as seven scripted review roles under written briefs in `.claude/agents/`, each judging work it did not author. The separation is of duties, not of judgement: these are automated roles sharing one model family, not independent parties, and the value is that none of them wrote what it checks. The names are names.

Faris	the knight	attacks tests until they prove they can fail
Adel	the just	traces every claim to evidence and re-derives it
Amin	the trusted	scans content and history by signature
Hasib	the reckoner	owns cloud resource lifecycle, creation to teardown
Ghareeb	the stranger	fresh clone, clean state, stripped environment
Rawi	the narrator	writes only from evidence, and has no shell
Fahim	the perceptive	questions the author on decisions and trade-offs

The tool scopes are part of the design: Rawi has no shell, so every number it publishes comes from a log something else produced. The briefs are written to be read on their own, and `docs/METHOD.md` describes the practices they enforce.

Body text is Arial 11, as the brief asks for it by name, embedded rather than substituted with a metric clone. Terminal output is set in a monospace face, because column-aligned output is unreadable in a proportional one — a deliberate departure from the single-font instruction, named here rather than left for a reader to notice.

9. Insights

The measurements that changed what I believed, in the order they would matter to someone running this infrastructure.

`docker kill` is recorded as operator intent, and restart policies decline to fire. Docker distinguishes an administrative stop from a crash, and `unless-stopped` deliberately does not act on the former. A recovery drill built on `docker kill` therefore exercises the wrong path while appearing to prove recovery. A genuine drill needs PID 1 to exit non-zero on its own.

`depends_on: service_healthy` is compose-time ordering only. A daemon restart ignores it: all three services came back at once, with the backend starting 15 ms before PostgreSQL. What converges the stack is the application's retry loop, and knowing which mechanism is doing the work changes what you would harden.

SQL Server Agent ships present in the Linux container, merely disabled. The folklore says it is unavailable there. It is in the Developer image and `MSSQL_AGENT_ENABLED=true` brings it up.

Enhanced RTMP carries HEVC, and MediaLive's RTMP input expecting H.264 is a property of the receiver. The constraint people attribute to the container belongs to the service at the other end. This matters when choosing a contribution transport, because it moves the decision from "what can carry the codec" to "what will the ingest accept".

HEVC's advantage over H.264 is bitrate-dependent and crosses over. VMAF 74.686293 against 75.093992 at 12 Mbps; 69.975861 against 69.023453 at 3 Mbps. Quoting the headline saving at contribution bitrates quotes it out of its range.

A read taken straight after a write measures the page cache, and the tell is a number above the link's physics. 9.5 GB/s over a 1 GbE path is roughly 76 times the ceiling. The honest figures are 26.0 MB/s cold and 24.3 MB/s with `O_DIRECT`.

`OOMKilled` must be sampled at the kill, because the replacement container resets it. Inspecting after recovery finds a fresh state object with the flag false, which reads as a clean run.

At the 5 GiB maximum part size, proving multipart requires deliberately smaller parts. A 23 MiB object becomes a single part and the ETag reads `-1`. The requirement to raise the limit and the requirement to demonstrate multipart are both satisfiable, and not in the same run.

A transport stream's mux rate is not the elementary stream's bitrate. A probe of a segment measures the container, which carries padding to a constant rate plus signaling. Comparing a probed container rate against a configured encoder rate compares two different things.

A test worth trusting has been seen to fail for the right reason. A restore drill compared two data fingerprints and passed because both had captured sqlcmd's `Changed database context to 'AppDb'.` banner instead of data — a comparison of two identical banners, which would have passed whatever the restore did. Mutating a check until it goes red is the only way to know it can.

A mapped network drive over a VPN does not reconnect at sign-in. Windows reconnects mapped drives early in the logon sequence, and a tunnel that starts as a service is not carrying traffic yet at that moment. The drive is recorded `Unavailable` and Windows does not retry. Measured 2m23s after a reboot: TCP 445 to the server reachable, credential present in Credential Manager, drive still `Unavailable` and not attached to the session. Re-establishing once the tunnel was carrying traffic succeeded with no password supplied. Two separate things have to be true — the credential has to be stored, because `net use` with an inline password authenticates that logon session only and stores nothing, and the reconnect has to happen after the tunnel is up. Where the drive must be present at sign-in, that reconnect belongs in a network-triggered task rather than in the logon sequence.

A guard should test the condition it cares about, not a file something else might create. An Ansible task setting a Samba password was guarded with `creates: passdb.tdb` — a file the package writes at install time. The task skipped on the first run and left an account with a Unix

identity and no Samba password: a playbook reporting success over an unusable account. Testing for the account in `pdbedit -L` is the check the task is actually about.

A tunnel gateway that masquerades changes what the far end sees. WireGuard NATs tunnel traffic onto the bastion, because the apps VPC has no route back to the tunnel subnet. A firewall rule at the far tier written against the tunnel range matches nothing; it has to name the gateway. `smbstatus` settles it — the session's Machine column reads the bastion, not the client.

Defaults must fail toward the disposable target. A tool that resolves an unset endpoint variable to production sends a debug script to a live account the first time someone forgets. The resolver here defaults to the local endpoint and requires an explicit opt-in to reach AWS.

---

10. Appendix — Requirement traceability

Every sub-requirement in the requester's wording, against what was delivered. Full detail in `docs/TRACEABILITY.md`.

#	Requirement	Delivered	Evidence
1.1	Protocol used and why	SMB 3.1.1 justified on the identity model; dialect 0x311 and an encrypted session read from the kernel	01-direction-a.log
1.2	Mounting steps	Real Windows share mounted with mount.cifs; round trip verified by Windows reading back what Linux wrote	01-direction-a.log
1.3	Persistent across reboots	fstab entry with _netdev, nofail and a mode-600 credentials file, replayed by mount -a at container start; proven across two full reboots with the share readable and no manual command	01-reboot.log
1.4	Monitoring mount and disk performance	diskstats, iostat, fio, node_exporter and Prometheus over both mounts, plus CIFS session and reconnect counters	01-monitoring.log
1.5	Alerting to guarantee service level	10 rules against a stated SLO, promtool clean; 3 driven inactive to pending to firing and back	01-alerts.log
1.6	Filesystem for 1 TB single files	XFS justified over ext4 and ZFS; 8 EiB ceiling measured, 1 GiB extent map across 3 allocation groups, grown online 2 GiB to 4 GiB	01-xfs.log
1.7	Reverse direction, Linux to Windows	Samba serving SMB 3.1.1 with encryption required and SMB1 disabled; executed end to end on the app tier, where a Windows client mapped the share and the server read the written file back on its own filesystem	05-vpn-smb.log
2.0	Apps on a private Docker network	Postgres, backend and nginx across two networks, data tier internal: true; 19 isolation checks	02-network-isolation.log
2.1	Automate as much as possible	One command to healthy with healthchecks and depends_on: service_healthy; idempotent migrations and seed	02-stack-up.log
2.2	Secret manager or environment variables	Both built and contrasted: docker inspect shows the env var in plaintext, the file secret only as a path	02-secrets.log
2.3	Recovery from unexpected failures	Five drills: crash, RAM exhaustion at OOMKilled=true exit 137, daemon restart, volume persistence, database outage	02-drill-b-oom.log
2.4	CPU, storage and RAM allocation	Limits and reservations on all three services, read back from the daemon: 2.25 CPU, 896 MiB	02-resources.log
3.0	Set up an MSSQL database	SQL Server 2022 self-hosted in Docker, pinned by tag and digest, healthy on 1433	03-stack-up.log
3.1	Create a database	AppDb in FULL recovery with page checksums, ONLINE	03-stack-up.log
3.2	Create tables and insert data	4 tables, 3 foreign keys, 7 checks, 11 indexes, seeded; constraints proven by negative tests	03-schema.log
3.3	Automated backups	Sidecar taking an unattended full backup at startup and running three tiers on their intervals with retention, each WITH CHECKSUM and RESTORE VERIFYONLY	03-backup-tiers.log
3.4	Restore from a backup	Full round trip to a byte-identical fingerprint, and STOPAT recovery landing between two committed markers	03-restore-roundtrip.log, 03-point-in-time.log

#	Requirement	Delivered	Evidence
4.1	Copy between buckets via multipart	Server-side UploadPartCopy engine and Node-RED flow; ETag suffix -5 against a single-PUT source	04-orphan-check.log
4.2	Raise upload size limit to maximum	5 GiB default with auto-raise past the 10,000-part ceiling; 1 TiB and 5 TiB plans asserted	04-part-sizing.log
4.3	Appropriate logging	One JSON line per event on a shared correlation id, with ranges, durations and progress	04-structured-logs.log
5.1	Three networks: public, apps, dbs	Three VPCs applied and configured; row written through all three tiers and read back	05-architecture.log
5.2	Three-tier architecture	4 instances across 3 tiers plus a bastion, 2 of 4 with a public address, egress via Cloud NAT	05-architecture.log
5.3	Proper network segmentation	Deny-by-default with service-account sources; boundary enforced at the routing layer, proven three ways with a positive control	05-architecture.log
5.4	Private VPN access to app and DB tiers	WireGuard on the bastion with a client tunnel established: handshake completed, counters nonzero both ways, app tier answering ICMP and 445 through it, and the same address unreachable with the tunnel down	05-vpn-smb.log
5.5	Terraform or similar	32 resources described, validated, formatted and planned clean	05-plan.log
5.6	Ansible configuration	Five roles through a dynamic inventory with ProxyJump; changed=0 on the second run across four hosts	05-ansible-idempotency.log
6.1	At least Full HD	1920x1080 locally and from MediaLive, measured with ffprobe on the AWS-produced segment	06-s3-archive.log
6.2	12 Mbps video, 192 kbps audio	12,652,330 bps overall locally with audio at 194,205 bps; AAC exactly 192 kb/s from MediaLive	06-encode.log
6.3	Justify bitrate with a BPP formula	BPP computed across four resolutions and six bitrates; 0.0965 bpp at 1080p60	06-analysis.log
6.4	HEVC codec	H.265 Main, 2-second GOP, encoded by MediaLive from an OBS feed, confirmed as hevc (Main) with stream type 0x24 at 60 fps	06-s3-archive.log
6.5	Record to S3 as .mxf or .ts	ARCHIVE output group writing MPEG-TS over s3ssl://; 36 segments from the OBS feed, one pulled back and probed	06-s3-archive.log
6.6	Document every step	10 AWS resources described and planned clean; encoder profile documented field by field; operator runbook against the deployed RTP input	06-terraform-plan.log